Wallet Infrastructure: Empowering the Next Generation of Dapps

1kx 0x91e2 nich December 12th, 2023 Mint

Wallet infrastructure is playing a crucial role in unlocking the web3 experience for the next generation of dapps.

So far, users have had to install additional software, source and fund it with a novel currency, and face unfamiliar confirmation screens before even making the first interaction in web3. Despite improved firewalling and a movement away from seed phrases, these hurdles are still points of high churn for decentralized applications.

The environment has been ripe for innovations that abstract technical layers

away, enabling intuitive onboarding to new financial, social, and gameplay experiences without compromising on the original ethos of self-custody and decentralization.

2023 has been a pivotal year for the wallet ecosystem, with account abstraction and developments across all layers of the stack shifting market structures and changing how we think about the relationship between users, dapps, and wallets.

This article covers:

- Account abstraction and its benefits
- An overview of wallet infrastructure and AA stack
- Emerging dapp/wallet development patterns and their implications
- Ongoing challenges and solutions to further explore

Account Abstraction: What, Why, and How

We can think of account abstraction as the decoupling of *account management* from *key management*. An account is an entity on the blockchain that can hold assets and have a transaction history. Signers (keys) are entities with the authority to perform actions on behalf of accounts.

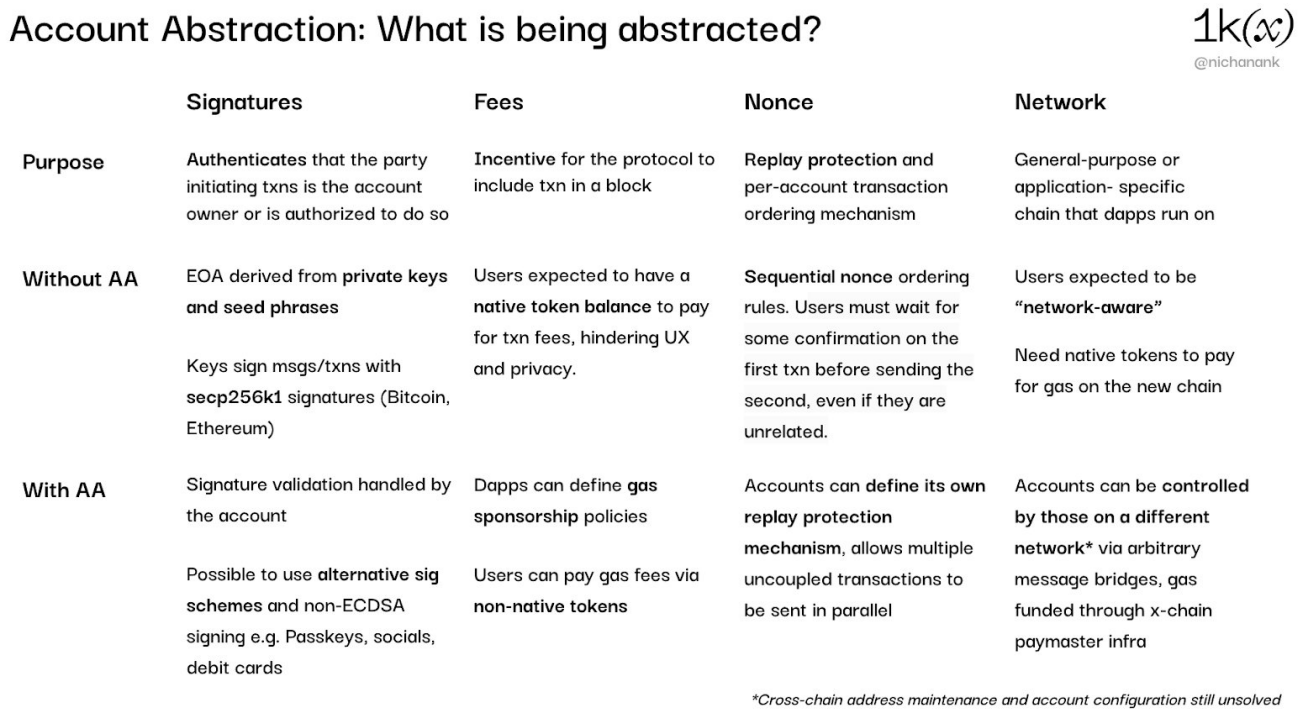
With traditional accounts (EOAs), a private key retains sole and total control over its associated account. The strict 1-to-1 mapping between the private key and the account means that:

- Users are limited to using dedicated key management solutions (e.g. Metamask, Ledger) when interacting with the blockchain.
- There is no path of recourse from losing a private key, and the key that controls an account cannot be switched out.
- All actions originated by that private key are treated as equal, from minting a free NFT to moving millions of dollars.

Account abstraction makes the account a smart contract with its dynamic logic for what key(s) can perform actions on its behalf, scope permissions, and do additional checks and balances according to the use case.



We can further break down its benefits by examining what is being abstracted.



Because the Ethereum protocol only recognizes EOA-originated transactions, account abstraction requires an offchain infrastructure to relay smart contract-

originated transactions to the chain.

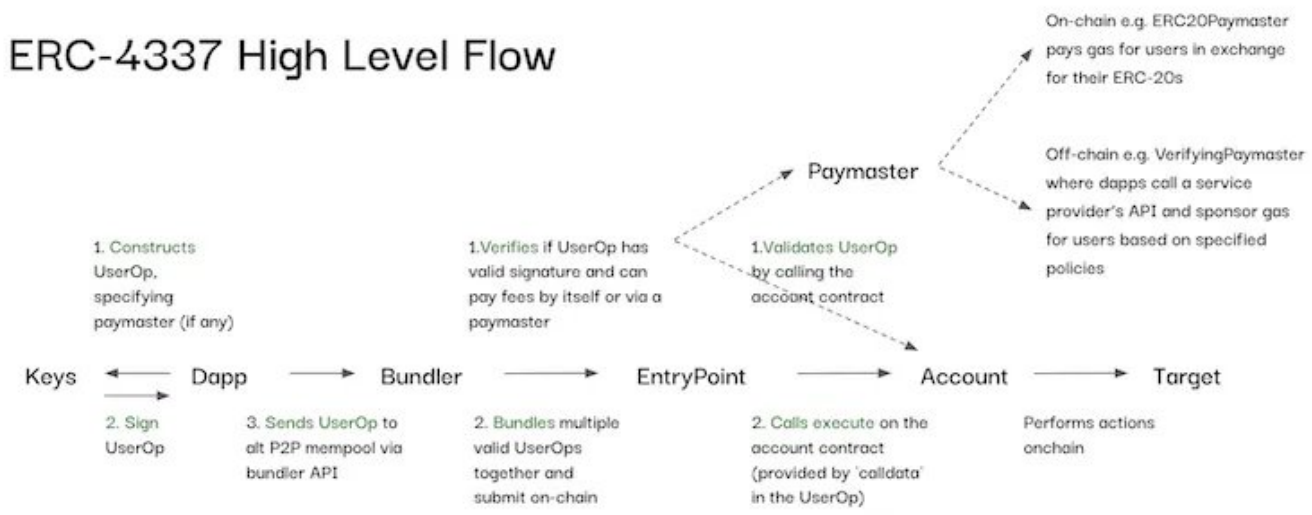
ERC-4337 was introduced in 2021 as a standardized way to do this without changes to the core protocol. However, some projects had been delivering on the benefits of AA long before the standard was fully fleshed out.

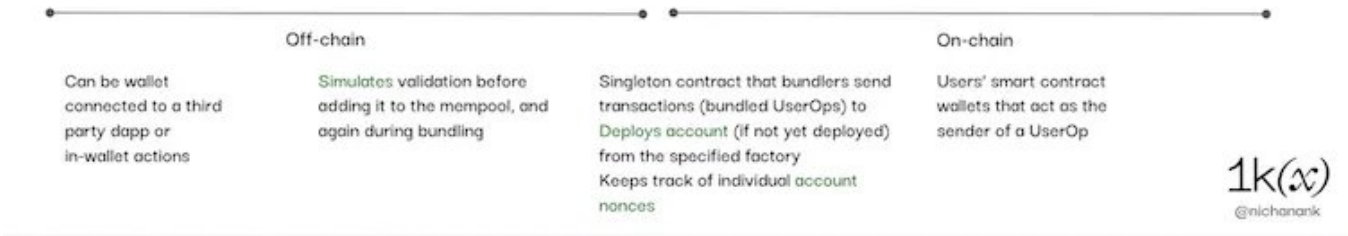
- Safe* multisig wallet launched in 2017 and has grown to secure $50B+ worth of assets for DAOs, businesses, and individuals alike
- Argent's mobile wallet has been powered by smart contract accounts since 2018
- Sequence wallet, launched in 2021, enabled Skyweaver to create and login to their smart accounts using their email and pay fees using non-native tokens

This required building and maintaining custom relaying infrastructure by the respective project.

Enter ERC-4337. The standard offers a decentralized and censorship-resistant alternative for the relay layer, defining an interface for accounts, paymasters, and signature aggregators to interact with third-party relayers via a shared alternate mempool of account-abstracted transactions ("User Operations").

Relayers ("bundlers") bundle multiple UserOps together into a transaction to send to a singleton EntryPoint contract, which subsequently verifies that the fees will be paid (by the account itself or through paymasters), and executes on the UserOps respective to the smart accounts.





We can contrast this with how validation and execution happen on chains that offer account abstraction natively and thus does not require extra relays (e.g. zkSync* and Starknet), as well as the recently published RIP-7560 proposal for native AA on Ethereum and its rollups.

Comparison of Account Abstraction Implementations

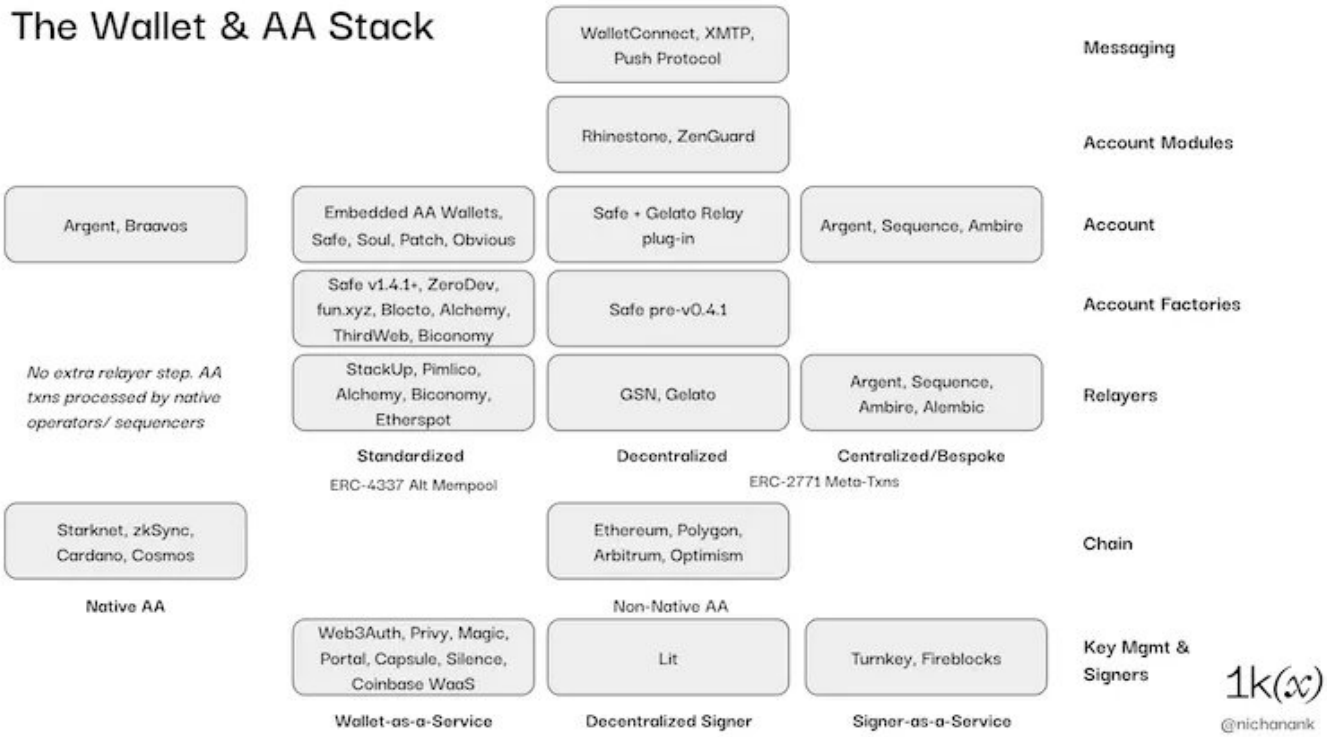
1k(x)
@nichanank

	ERC-4337	RIP-7560	zkSync Era	Starknet
Transactions & Mempool	**Signed UserOps** (pseudo-txn) sent to alt mempool to be picked up by Bundlers.	**Signed AA_TXN_TYPE txns** sent to alt mempool to be picked up by Builders.	**Signed L2 transactions** (EIP-712) sent to zkSync's protocol mempool.	**Signed L2 transactions** sent to Starknet's protocol mempool.
Components	**EntryPoint**: calls AA methods on the account and paymaster e.g. validate, execute, postOp. Also keeps track of account nonces. **Bundlers**: bundles UserOps and relays to the EntryPoint **Paymasters**: sponsor txns or allow token payment	**Bundlers/Builders**: processes AA_TXN_TYPE txns and are incentivized with builderFee **NonceManager**: ensures account nonces are unique but doesn't require them to be sequential **Paymasters**: sponsor txns or allow token payment	**Bootloader**: calls AA methods on the account and paymaster e.g. validate, execute, postOp **NonceHolder**: keeps hold of deployment & protocol txn nonces **Operators**: rolls L2 txns together, computes zkp of state transition, and affects the state transition by interacting with the rollup contract **Paymasters**: sponsor txns or allow token payment	**Sequencers**: put txn in order, produce and posts proof to L1. **Starknet OS**: calls validate, execute, postOp. to ensure sequentially increasing nonce, signatures, fee payment to sequencer *Paymasters: sponsor txns or allow token payment*
Validation	**EntryPoint contract** checks validity of nonce, signature (by calling validateUserOp), and ability to pay fee (by account/paymaster). Misconfigured bundlers can accept UserOps with insufficient fees - operating at a loss	**Bundlers/Builders** processes the validation frame of AA_TXN_TYPE. Executes on nonce validation and increment, sender account contract deployment (if applicable), signature validation, and paymaster validation (if applicable)	**Native Protocol** checks validity of nonce, signature (Bootloader calls validate Transaction), and gets payment (by account/paymaster).	**Native Protocol** validates txn and collects fee, so the sequencer is guaranteed to receive the payment, even if the txn fails later during execution.
Execution	**EntryPoint** sends UserOp callData to the account, which should have an execute function Calls postOp on paymater with custom logic for fee payment	**Bundlers/Builders** processes the execution frame of AA_TXN_TYPE. Executes on paymaster post-execution frame (if applicable).	**Bootloader** calls execute Transaction on the account. Optionally call postOp on paymaster to handle e.g. gas refunds	**Starknet OS** calls execute on the account. *Call postExecute on paymaster;*

In March 2023, the 4337 EntryPoint contract was deployed to mainnet. Its community has been immensely successful in getting developers to participate in the account abstraction movement.

This ushered in a wave of new infrastructure and service providers to the wallet ecosystem, and pushed existing projects to ensure their business strategies and product suites continue to address the needs of application developers looking to leverage AA to offer their users a seamless web3 experience.

Wallet Infrastructure & AA Stack



Signers & Key Management

Signers & Key Management infrastructure is responsible for generating and securing public key pairs used to sign messages, transactions and UserOps. The most straightforward example here are traditional EOA wallets, but wallet-as-a-service providers have emerged to enable seedless onboarding and wallet management via alternative auth methods like social and email.

Under the hood, these services either store key material in HSMs like AWS KMS which only the user can access via their auth credentials (Magic, Turnkey), or operate under some SSS/MPC scheme (Privy, Web3Auth, Portal, Capsule) to protect the materials.

Lit* improves upon this server-side key store design by decentralizing the keys. Each node in the network stores a share of a ECDSA private key generated via a DKG algorithm, all operations taking place in encrypted virtualization. Arbitrary authentication rules can be assigned to the key pair, giving the application or user complete control over what interactions are allowed, and impose for example spending limits. The network can further be leveraged by 2-of-N MPC

wallets as a backup and recovery option.

This year, there has been rapid experimentation to leverage Hardware Signers and Passkeys as a signer to an account to **give users key management out-of-the-box with modern mobile or desktop devices**. These signers work natively with biometric authentication (e.g. FaceID, TouchID) to give additional security with familiar UX.

- **Hardware Signers** utilize isolated subsystems like the iPhone Secure Enclave and Android Titan HSM to generate keys and sign messages, guaranteeing hardware-level security. Because the keys cannot be extracted from the device, this is most powerful in tandem with additional recovery methods or as part of a 2FA system.
- **Passkeys** are a standard for passwordless authentication built on top of WebAuthn. Here, a keypair is generated in the device's operating system, and can be synchronized across devices through services like iCloud, so recovery is possible if the user has opted in to do so.

One limitation here is that Passkeys and Hardware Signer-produced signatures are not natively recognized by chains like Bitcoin and Ethereum. They use the secp256r1 (R1) elliptic curve, while these chains operate on the K1 variation. While there is ongoing work to trustlessly and efficiently verify the R1, some Passkey-enabled products are going through services like Lit and Turnkey to produce a K1 signature once the user has authenticated with their key.

A standard to watch here is EIP-7212, which proposes to add the R1 curve directly to the EVM as a precompiled contract so every modern device can natively sign transactions without third party services or middlemen.

As the volume of account-abstracted transactions grows, signature aggregation using BLS signatures could lead to smart account fees to be cheaper than EOAs on L2s. 4337 defines an interface for Aggregator helper contracts which validates a single aggregated signature approving multiple UserOps as opposed to validating each one separately.

Relayers

Relayers (e.g. 4337 Bundlers) relay transactions or UserOps to a mempool. On chains with native AA, network operators and sequencers play this role, removing the need for external dedicated relayers.

Similar to how there are multiple client implementations for Ethereum (e.g. geth, erigon, reth), the 4337 ecosystem has multiple bundler implementations in different languages, making the network more robust against vulnerabilities of a single implementation. The 4337 specs include a test suite to ensure bundler compatibility across the network. Implementers include Stackup (Golang), Pimlico, Biconomy, Etherspot (Typescript), Candide (Python), OKX (Java), and Alchemy (Rust).

The incentive model for bundlers is similar to block-builders, taking fees from user operations it bundles instead of transactions. In practice, bundlers need an API into the block-builder to see the current block and create a bundle that is valid against that block and thus should be considered as part of the block builder.

As 4337 traction grows, we can expect that builders will also be bundlers as this hybrid will be more profitable than a builder alone as they can select from both the transaction and UserOps pools.

Paymasters

Paymasters enable fee abstraction by allowing dapps to sponsor gas for users, allowing users to pay fees using non-native tokens, or settling offchain via traditional payment rails. Paymaster services have 2 main components:

- **A gas policy manager** for developers to define conditions under which they will sponsor gas. This can be scoped to the entire project, or on a per-contract or per-wallet address basis. Developers can also define how they want to rate-limit gas sponsorship e.g. by gas price, # requests, or $ amount sponsored/month. Gas sponsorship costs are usually rolled into the developers' monthly invoice from the service provider with a ~5% surcharge

for the sponsored amount.

- **A paymaster smart contract** validates whether a given transaction is eligible to be covered, either based on onchain state like account balances or offchain gas management policies. The paymaster contract holds a balance of native tokens that is used to pay for gas, and may contain price oracle logic that periodically checks the exchange rate between the payment token (e.g. USDC) and native token (e.g. ETH).

Paymasters can be categorized into onchain or offchain:

- **Onchain Paymasters** (e.g. ERC20Paymaster, StablecoinPaymaster) only rely on onchain state to verify whether a transaction is able to be covered by the paymaster or not. This means that certain paymasters, like those that accept gas payment in ERC-20s, can be made permissionless, with the caveat that the paymaster has to be approved by the account to transfer out the payment tokens. Paymaster contract admins can withdraw tokens and convert back to native to refill the contract, set a markup on top of the ERC20 price, set a threshold for the price difference for which the paymaster updates the ERC20 price for the next UserOp, or manually update the price.
- **Offchain Paymasters** (e.g. VerifyingPaymaster) involves interactions with a service provider's paymaster API to sponsor the UserOp. The offchain service checks eligibility and signs off on the transaction using the paymaster's keys. While this solution is permissioned, offchain paymasters come with the benefits of gas savings from minimizing onchain checks. Gas policies can be made more granular, and take offchain activity such as Discord activity into account.

Account Factories & Frameworks

Account Factories & Frameworks provide "headless" smart account implementations and SDKs that dapps and wallet clients can build on top of, creating self-custodial embedded accounts on behalf of their users. The accounts themselves are smart contract wallets that have their own signature validation, execution, and replay protection (nonce management) logic. Owners authorize user operations originating from their smart accounts using their key.

At a high level, smart account vendors provision 3 core things:

- **A core implementation** for a smart contract wallet, with its own logic for how transactions are validated, executed, and any additional actions to perform before and after the execution. It also contains logic for how additional features can be added to the wallet via native and third-party modules.
- **A factory contract** that deploys fresh instances of the wallet implementation, initiated with the initial signer for that account. Under ERC-4337, dapps can create smart accounts for their users by specifying the factory contract address from their provider of choice.
- **An SDK** that gives developers plug-and-play customizability for the smart accounts they are creating. This can include different signing options, on/off-ramping, and relaying technologies.

Under ERC-4337, the `sender` field of a UserOp refers to the smart account under which the transaction is being made. If the account hasn't yet been deployed, the EntryPoint deploys the account from the factory contract specified in the `initCode`. User's keys can be used to claim the smart account to make subsequent dapp interactions.

Account providers like Safe, Zerodev, and Biconomy integrate with key managers and authentication infrastructure to give dapps optionality on how they want users to manage their smart accounts. For example Safe's Web3Auth integration enables users to use their accounts using social or email, and Zerodev's integration with Turnkey gives the option to manage accounts with Passkeys.

Safe is most known for its battle-tested smart wallet product widely used by individuals, teams, and DAOs. To date there have been 5m+ Safes deployed across 12+ chains, executing over 22m+ transactions. Prior to v1.4.1 (launched July 2023), developers were already able to use Gelato relay to enable gas-abstracted transactions. This combination currently powers crypto debit card products like Gnosis Pay and BasedApp where users are able to purchase from any vendor which accepts Visa using funds in their Safe. v1.4.1 brings ERC-4337 support through modules to give additional optionality in relay providers.



ZeroDev is a smart account provider launched earlier this year built for ERC-4337 from the start. Zerodev aggregates multiple bundler providers to abstract UserOp relay services, and exposes a gas manager dashboard where devs can define the scope and rate-limiting logic for which to sponsor fees for users. Zerodev and Biconomy (which also runs its own bundler network) currently dominate the market share for 4337-enabled accounts.

Alchemy's AccountKit features a 4337-compliant smart account implementation "LightAccount", which is based off of the EF's implementation and adds EIP-1271 support (validating signatures originating from smart contracts) along with ownership transfers, key rotation, and namespace storage.

Account Modules

Account Modules are smart contracts that act as installable components to a smart account. Although module infrastructure is still in very early stages, we envision that modules will be discoverable and installable by:

- **Developers**: embedded smart accounts can come with "pre-installed" modules as decided on by the dapp developer, constructing a starter wallet with features customized for their use case

- **End users**: wallet interfaces can expose a “module store” where users can discover new features and add them to their wallet

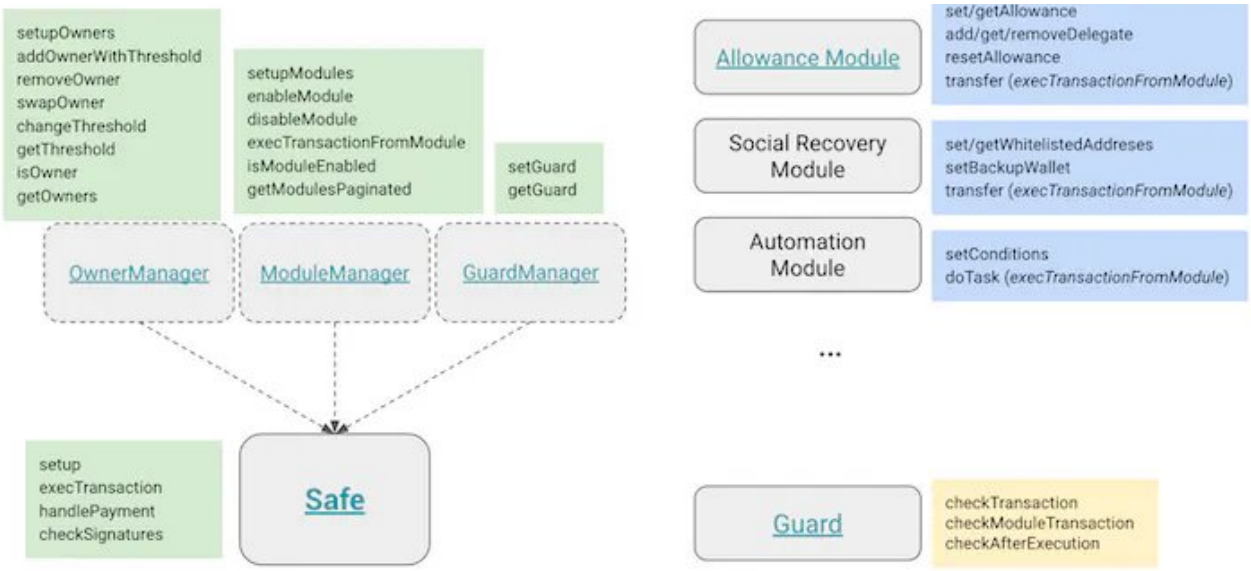
With AA formally separating how UserOp validation and execution is handled, modules can contain validation-only or execution-only logic.

- **Validators**. Called during the validation phase of a UserOperation.Their primary function is to verify the signature of a UserOperation and determine whether it is valid and should be executed. Examples include multisig, ECDSA, passkeys, multichain validation, and session keys. Session keys enable dapps to sign on behalf of the user to simplify UX, behaving like temporary private keys with customizable permissions and expiry time.
- **Executors**. Called during the execution phase of a UserOperation. They extend the execution logic of the account and allow for a more diverse set of actions that it can natively perform. Examples include automated actions that are triggered outside of the regular ERC-4337 execution flow such as automatic token swaps when the price reaches a certain threshold.
- **Hooks**. Run pre- or post-execution and enforce controls against the account. For example, a hook could run post-execution and revert any transactions that meet certain criteria to create increased security for the user.

While some wallets like Candide have developed modules their users can install directly, we anticipate a rich ecosystem of third-party modules discoverable in an app-store like interface of their wallets, or composed into a “starter” embedded wallet by dapp developers.

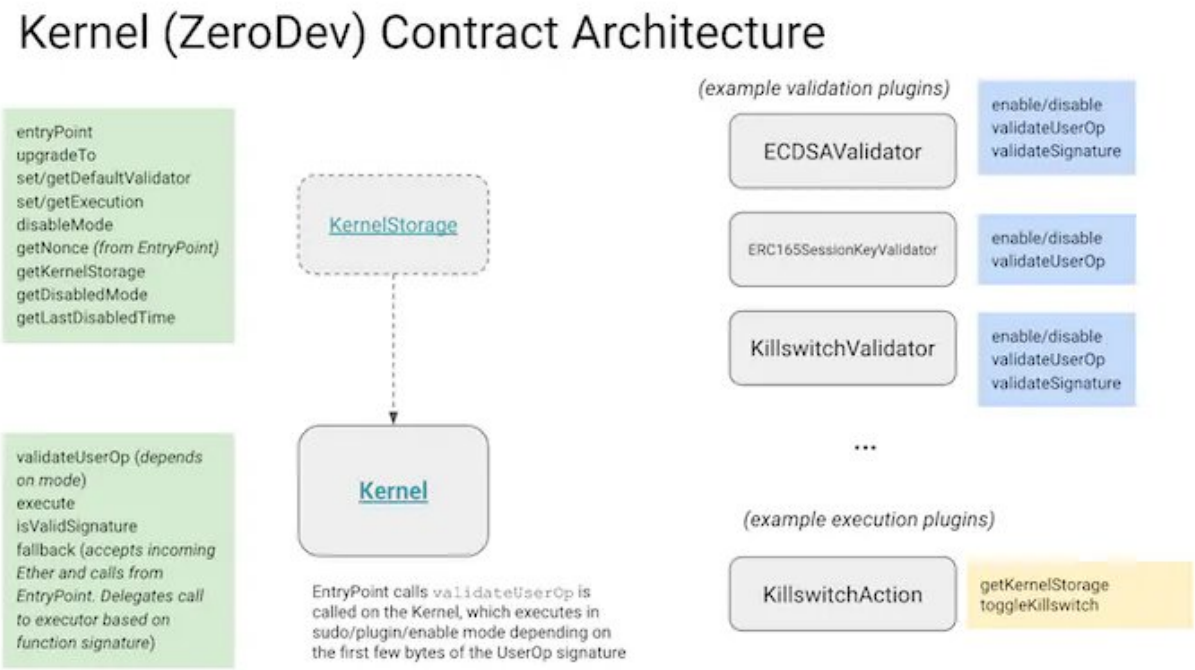
Smart account frameworks are already designed with modules in mind. Safe’s core contract handles module *management* logic for adding and removing modules from the account, but leaves actual module-related logic and storage completely scoped to a separate contract. This separation of concerns mitigates the risk of third party modules overwriting the same state, compromising the security and expected behavior of the account.





Safe{Core} Protocol introduces an open framework with modules, hooks, a manager, and registries aimed to foster an composable smart accounts ecosystem inspired by the learnings from Safe's wallet product.

ZeroDev explicitly classifies their modules ("plugins") as validation or execution. Executor modules are designed to be paired with validator modules, allowing custom functions to be "routed" through different validators. For example, an "NFT transfer" function that only allows NFTs to be transferred via 2FA.



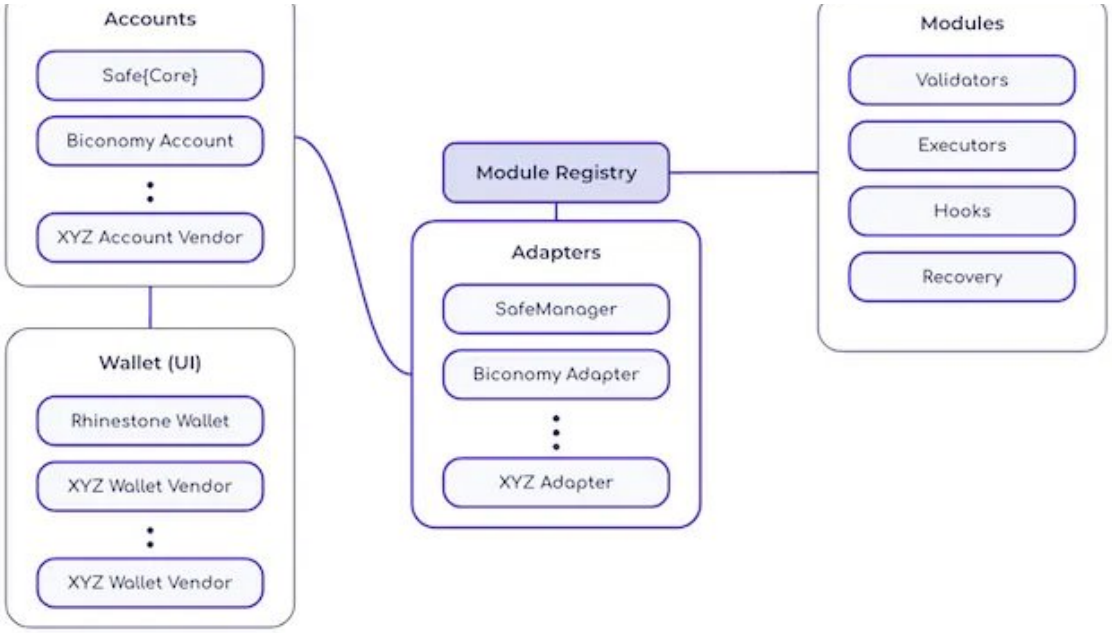
Some considerations in building towards a robust ecosystem of modular smart accounts:

Interoperability. With multiple smart account vendors each with their own approach to how third parties can add new features to accounts, module developers are on a trajectory towards vendor lock-in or having to manage the technical overhead of developing the same features to be compliant with multiple account implementations. Some solutions to mitigate this:

- ERC-6900 for modular smart contract accounts and plugins defines interfaces for modular smart contract accounts (MSCAs), modules ("plugins) allowing any standard-compliant account implementations and plugins to be interoperable with each other.
- Rhinestone's* ModuleKit for building and testing smart account modules provides templates and frameworks for testing modules against different account implementations, a library of integrations (e.g. DeFi protocols), pre-built conditions for execution, and security automation to parse through module code and flag security vulnerabilities.

Security. Giving users the ability to install third-party software into their wallets comes with important questions for how interfaces should curate and expose relevant information about modules.

- EIP-7484 provides a means of verifying the legitimacy and security of independently built smart account modules. Here, a registry allows auditors to make attestations about the security of those modules. Frontends and smart accounts can query the registry to pull in attestation data, verifying that a module is safe to use. See the Rhinestone registry for a reference implementation of this.
- More broadly, EIP-7512 aims to create a standard for an onchain representation of audit reports parsable by smart contracts to extract relevant information around who performed the audits and what standards have been verified.



Discoverability. Registries can be exposed and queried by smart account platforms and wallet interfaces to be installed by developers or end users.

The ability to extend wallet functionality turns accounts into developer platforms and new distribution channels for web3 products and services. We are already seeing this with Metamask Snaps which allows users to customize their browser extension wallet with security alerts (through WalletGuard), privacy features (through Nocturne), and interoperability with non-EVM chains such as Starknet and Bitcoin.

When Chrome enabled an ecosystem of developers to extend browser functionality through extensions, this propelled them to market dominance over incumbent walled gardens despite being a decade younger. Although it's hard for most of us to picture what the wallet experience might look like when modular accounts become mainstream, the tracks are already being laid for permissionless innovation to take its course.

Emerging Development Patterns & Their Implications

The evolved wallet stack means that:

- **Developers can create non-custodial accounts for their users in the context of their apps**, and will look to tools like connector SDKs to give them optionality in how to construct the end-to-end onboarding journey.
- **Embedded wallets are a new wallet category**, each vendor with its own features and tradeoffs in terms of account portability, customizability, and trust assumptions.



If you played around with Ethereum dapps in 2018, you may recall getting a Metamask popup as soon as you load the site. This was due to a lack of good UX practices around connecting wallets and dapps, and developers often resorted to hardcoding checks to see if a user has an extension wallet installed using the browser's `window.ethereum` object. This resulted in unpredictable behavior if users had multiple extension wallets installed, forcing users to choose one and creating a less competitive market for wallets.

WalletConnect* communications protocol emerged to let users connect any wallet to any dapp, alongside with Web3Modal, a library that wraps buttons and modal components to let users choose which wallet they wanted to use the dapp with.

Today, Web3Modal is one of multiple wallet connector libraries like RainbowKit, Web3Onboard, and ConnectKit that streamlines the wallet detection and wallet-based authentication process for dapp developers. **These libraries offer out-of-the-box theming options, wallet search features, and screens that redirect users to install wallets if they don't already have one**.

More recently, EIP-6963 was finalized as an alternative wallet discovery mechanism to `window.ethereum`, allowing dapps and injected scripts provided by extensions to communicate in a predictable manner. **Thanks to the standard, users now have more optionality in choosing an extension wallet of choice to connect to dapps, and opens a more competitive market for wallets**.

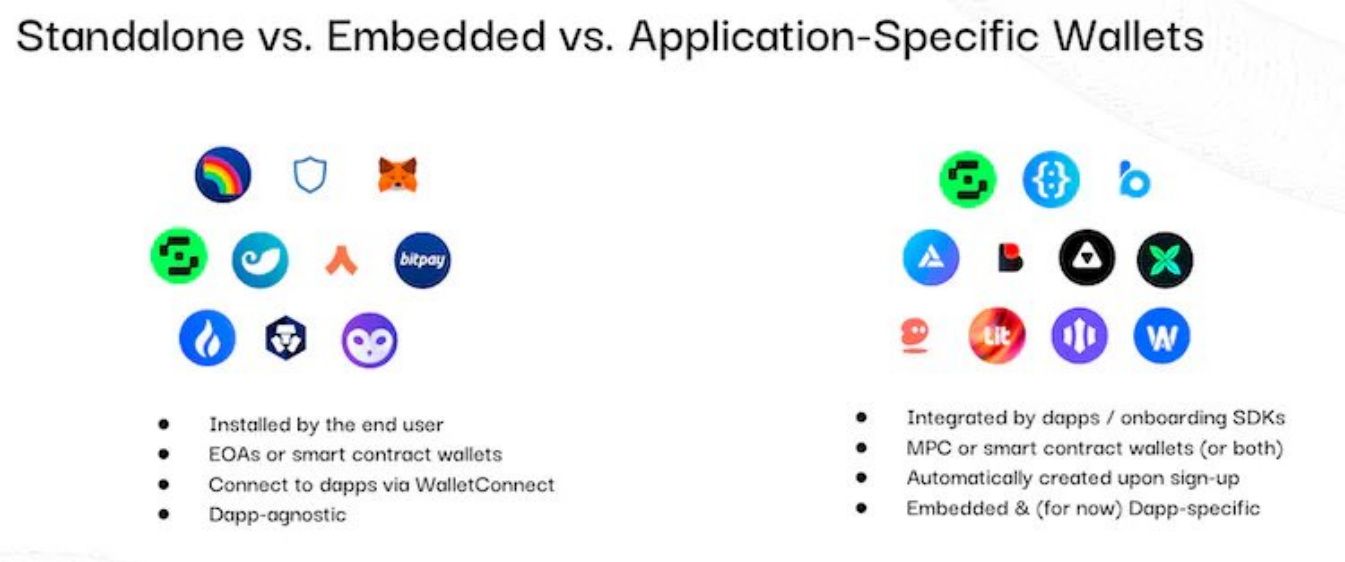
While connector libraries have significantly improved DevEx and UX,, the expectation that users already have or will install additional software to interact with dapps still poses a high barrier to adoption.

We are already seeing a glimpse of what the future of dapp onboarding UX looks like, as the next generation of wallet libraries, which we will call here fully fledged "onboarding SDKs" gain steam. In addition to wallet-based auth, these SDKs give alternative sign-up and login options such as email, socials, SMS, and **creates embedded wallets for users without requiring them to install additional software or navigate away from the dapp**.

Developers can integrate connectors offered by key providers directly (e.g. Magic, Privy, Web3Auth) or use those that wrap multiple services (e.g. Dynamic, Thirdweb, 0xPass, Web3Modal) to give plug-and-play optionality on the authentication options they want to expose and the type of wallet they want to create, completely customizing the onboarding flow. Onboarding SDKs can also integrate with smart account providers to create embedded smart wallets, offering further UX enhancements like gasless transactions and on/off-ramps.

With increased adoption of "headless" wallets and a movement towards embedded wallets, what used to be a clear line of separation between wallets and dapps is beginning to blur.

Standalone Wallets vs. Embedded Wallets vs. Application-Specific Wallets



Web3 users today are used to interacting with dapps through standalone wallets like Metamask or those offered through WalletConnect, accruing their assets and onchain footprint to one or a few accounts.

As more dapps look to onboard users through embedded wallets and multiple vendors available, account management quickly becomes complicated for both dapp developers and end users. Dapp developers will have to evaluate and manage multiple vendors while trying to avoid lock-in. For end users, creating a new wallet per dapp will lead to a fractured asset and identity management experience.

While app-specific wallets are desirable for certain use-cases such as games, there are many instances where users might onboard to their first dapp, create an embedded wallet with their web2 signers or Passkey, and want to use the assets they accrue in that account on another dapp by logging in with the same signer.

- Capsule is a MPC-based embedded wallet provider that features wallet portability across dapps that use their service, giving users access to an existing wallet using the same email login. **We can anticipate this to soon be a table-stakes offering across WaaS providers**.

- Moonchute helps users manage multiple smart accounts in the interim. Their unified account manager is an app and API for discovering smart wallets created by a given signer, allowing users to manage assets from multiple accounts in one place.
- ERC-7555 proposes a SSO-inspired standardized interface and request/response pattern for applications to discover user accounts created using alternate signing schemes. Here, the application would redirect the user to a given provider's URI (which could be a self-hosted domain), and parse the response for a signer and associated smart account address.

Migrating from EOAs

Another outstanding challenge of AA is a seamless way for existing users, who already have assets and onchain history accrued on multiple EOAs, to migrate to smart accounts.

- EIP-7377 proposes an in-protocol mechanism to allow EOAs to send a one-time transaction which deploys code at their account, effectively "upgrading" an EOA to a smart wallet.
- Aarc aims to solve asset migration for dapps and end users. Their UI and SDK indexes the assets and permissions of a particular source address, and allows users to select the assets that they want moved to any destination address, which could be a smart account, another EOA, or embedded MPC wallet created with social login. For dapps with existing users who are used to the standalone wallet flow, Aarc offers a solution to streamline the migration process as they look to add embedded wallets or AA features to their product.

AA implications for multichain account management

Given the momentum of AA and L2 activity, we can anticipate a future where smart accounts become mainstream over EOAs with users having assets across multiple chains.

One UX advantage of EOAs is that users automatically have access to the same address across different EVM chains using the same private key. The drawback

being that it is not possible to change the keys that control a given address, and all funds can be lost if the user loses their private key.

Because account abstraction separates the key store from the asset store, it is possible to rotate keys for a given account without having to migrate funds while keeping the same address. Smart accounts are capable of maintaining the same address across chains using CREATE2, giving users access to the account even if the contract hasn't been deployed on a given chain yet using the same initial verification key and account implementation.

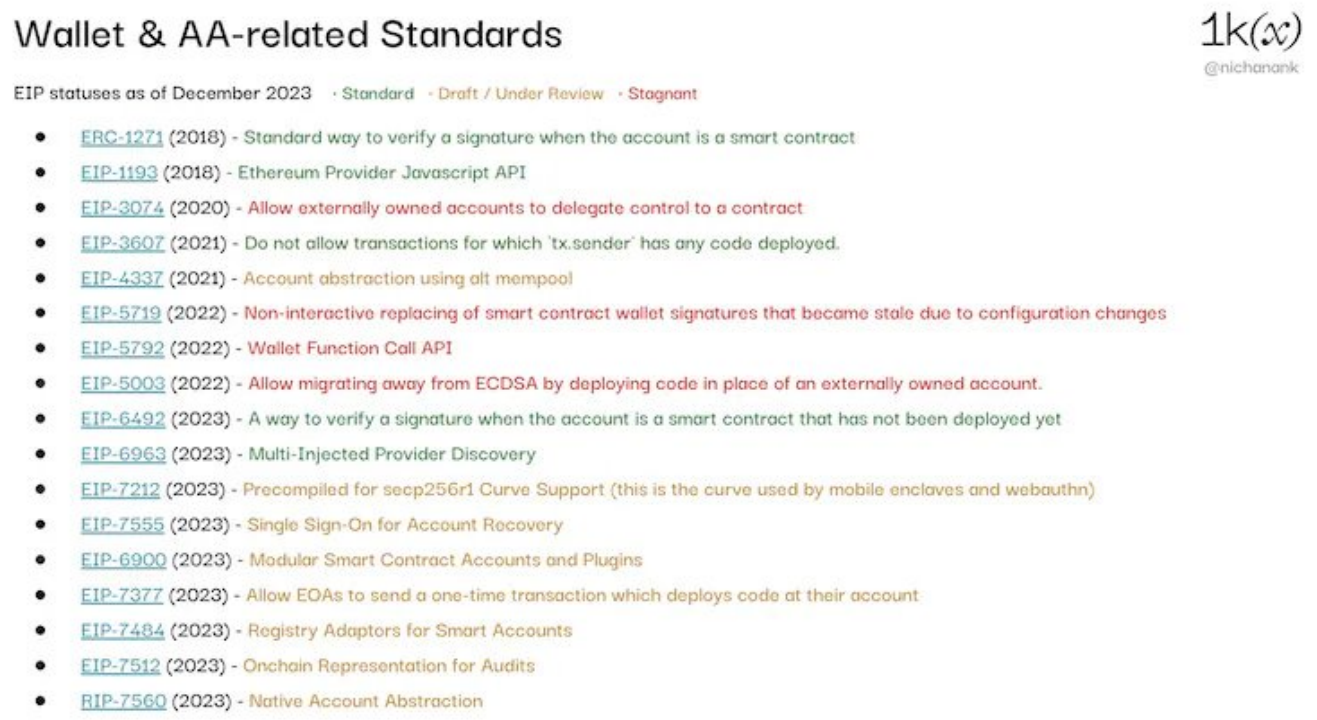
However, maintaining the same address across chains may be an anti-pattern in the long term.

- **CREATE2 is only possible on chains with EVM bytecode equivalence**. In a multichain world with zk-Rollups (e.g. zkSync) with slight deviations to the EVM, this approach will not suffice.
- **We can expect that keys needed to access various accounts are going to change over time** as wallets expose additional signing and key rotation features. Under the current setup, this can quickly cause state drift of account permissions across chains, as the changes to signers on an account on one chain doesn't automatically propagate the new permissions to those on other chains.

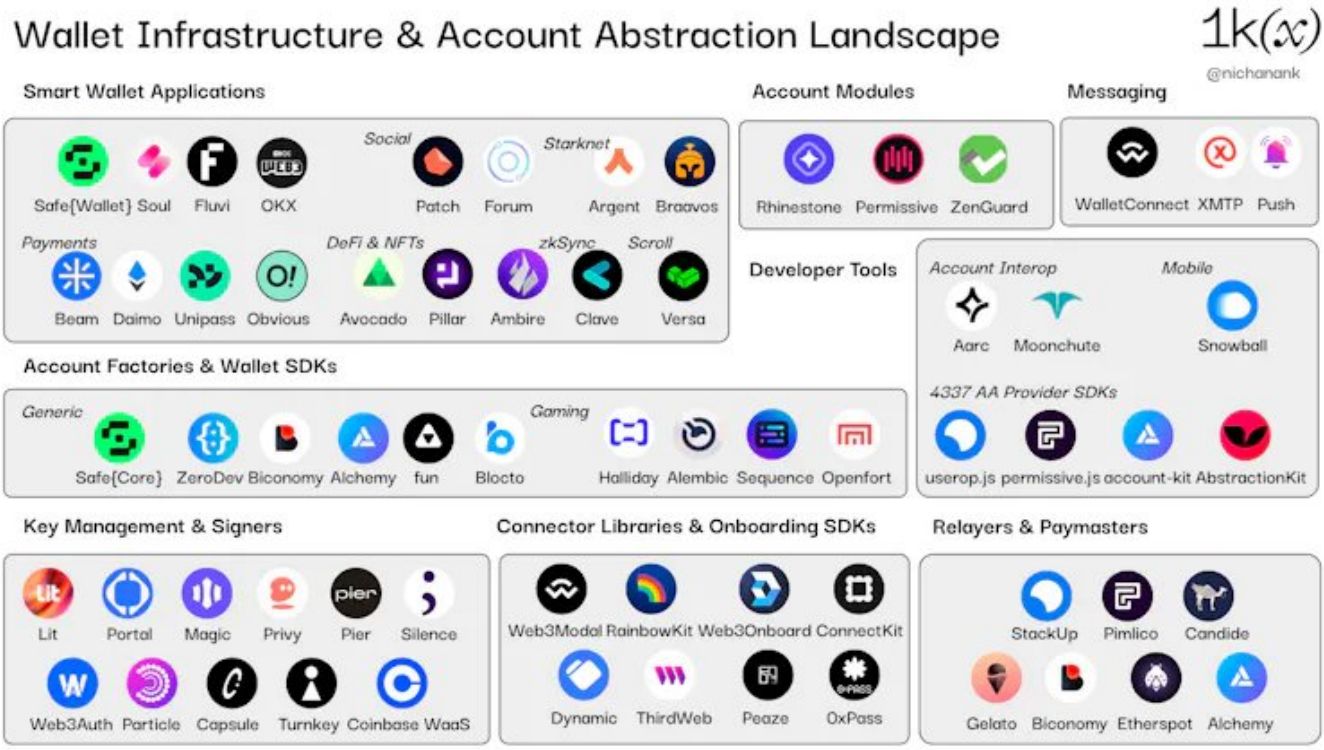
Longer term solutions that have been proposed for multichain AA include:

- A **dedicated key store contract** that a user account across chains read from when checking permissions. See an implementation of this from Soul Wallet.
- Using **ENS multichain resolvers** as an abstraction layer for different addresses.

Cross-chain account and signer management is still an area under active research. The end goal here is that ***a user can change the keys that have access to multiple accounts on different chains without making an extremely high number of transactions***.



Conclusion & Closing Thoughts



- Account abstraction is a movement to decouple signers from accounts by making contract-based accounts (instead of EOAs) as first-class entities on the blockchain, giving users flexibility in key management and account permissioning.

- ERC-4337 as a standard for relaying smart account-initiated transactions has catalyzed the evolution of wallet infrastructure to accommodate AA, giving rise to new market structures, wallet categories, dapp development and user onboarding patterns.
- The wallet stack has been unbundled into signers, relayers, account providers and account modules giving developers optionality in how they want to customize the end user experience. This comes with the additional overhead of evaluating each provider on their tradeoffs in terms of gas overhead, censorship resistance, vendor lock-in, and interoperability.
- Migration paths from EOAs and account abstraction in a multichain context are still areas of ongoing research. We expect to see the first implementations of proposed solutions in the coming year.

We believe these developments have significant implications across the ecosystem:

- **For new users, wallets are no longer the sole entry point into web3**. Applications will have markedly improved onboarding through embedded wallets, gasless transactions, and in-wallet onramps.
- **For current users, onchain experiences will become more seamless** as apps leverage features like session keys. Power users have more granular control over account permissions and additional wallet features through modules.
- **For developers, modular account infrastructure turns wallets into operating systems**. Module markets are a new permissionless distribution channel for web3 products and services.

Wallet infrastructure will continue to catalyze web3 adoption. We at 1kx are proud to have backed teams who have pioneered on the ideas discussed in this article, and will continue to monitor the space for what's to come.

If you are working on solutions in this space or have additional thoughts on the topic, please reach out @nichanank - would love to chat with you.

Many thanks to David Sneider, John Rising, Konrad Kopp, Kurt Larsen, Marc

Sednaoui, Dogan Alparslan, Vivian Phung, Derek Rein, Tom Terado, Diana Biggs, Mel Quarto, and pet3rpan for reviewing drafts of this.

**denotes 1kx portfolio companies*

Subscribe to 1kx
Receive the latest updates directly to your inbox.

Enter email address

Subscribe

Mint this entry as an NFT to add it to your collection.

Mint

Verification
This entry has been permanently stored onchain and signed by its creator.

ARWEAVE TRANSACTION↗
xw4VU24cOXpTLtB…M9wv5DQVSRN7C9g

AUTHOR ADDRESS↗
0xdbF14da8949D157…EEE62412b210900

CONTENT DIGEST
vrFC1TS9JwksV7_…Oz3xd0y3YWzDL7Q